

Integrated Tourism Booking and Travel Management System

Aditya kundu^{#1}, R Y Priyanka^{#2}, Sravya Surya^{#3},

Amrita School of Computing, Amrita Vishwa Vidyapeetham, Bengaluru, Karnataka, India

bl.sc.u4aie24004@bl.students.amrita.edu

bl.sc.u4aie24038@bl.students.amrita.edu

bl.sc.u4aie24048@bl.students.amrita.edu

I. Problem Statement

Tourism services such as hotels, tour packages, activities, destinations, and travel bookings are often managed across separate systems. This makes it difficult to maintain consistent information regarding availability, pricing, customer bookings, payments, and reviews. Such fragmentation can lead to data redundancy, inconsistent records, booking conflicts, and inefficient retrieval of tourism-related information. Users may also need to interact with multiple platforms to plan and manage different aspects of their trips.

Therefore, there is a need for a centralized database-driven Tourism Management System that can efficiently store, organize, and manage tourism information and booking transactions while maintaining data integrity, consistency, and efficient data retrieval.

II. Objective

The main goal of the Tourism Management Database System is to design and implement a centralized relational database that efficiently manages tourism-related information and booking transactions while ensuring data integrity, consistency, and efficient data retrieval.

The specific objectives are to:

- **Centralize tourism data** by storing information about destinations, hotels, rooms, activities, tour packages, customers, bookings, and payments in a single database.
- **Reduce data redundancy** by organizing the database using appropriate relational structures and normalization techniques.
- **Maintain data consistency and integrity** by enforcing primary keys, foreign keys, constraints, and appropriate relationships between entities.
- **Manage bookings efficiently** by tracking availability, reservations, cancellations, and booking details while minimizing booking conflicts.
- **Establish relationships between tourism entities**, such as customers, bookings, hotels, rooms, activities, destinations, and payments.

- **Enable efficient data retrieval** through SQL queries for searching destinations, checking availability, viewing bookings, calculating payments, and generating useful reports.
- **Provide centralized administrative management** for adding, updating, deleting, and monitoring tourism services and customer transactions.
- **Improve overall tourism service management** by providing a reliable and structured database system that supports accurate and efficient decision-making.

III. Scope of the System

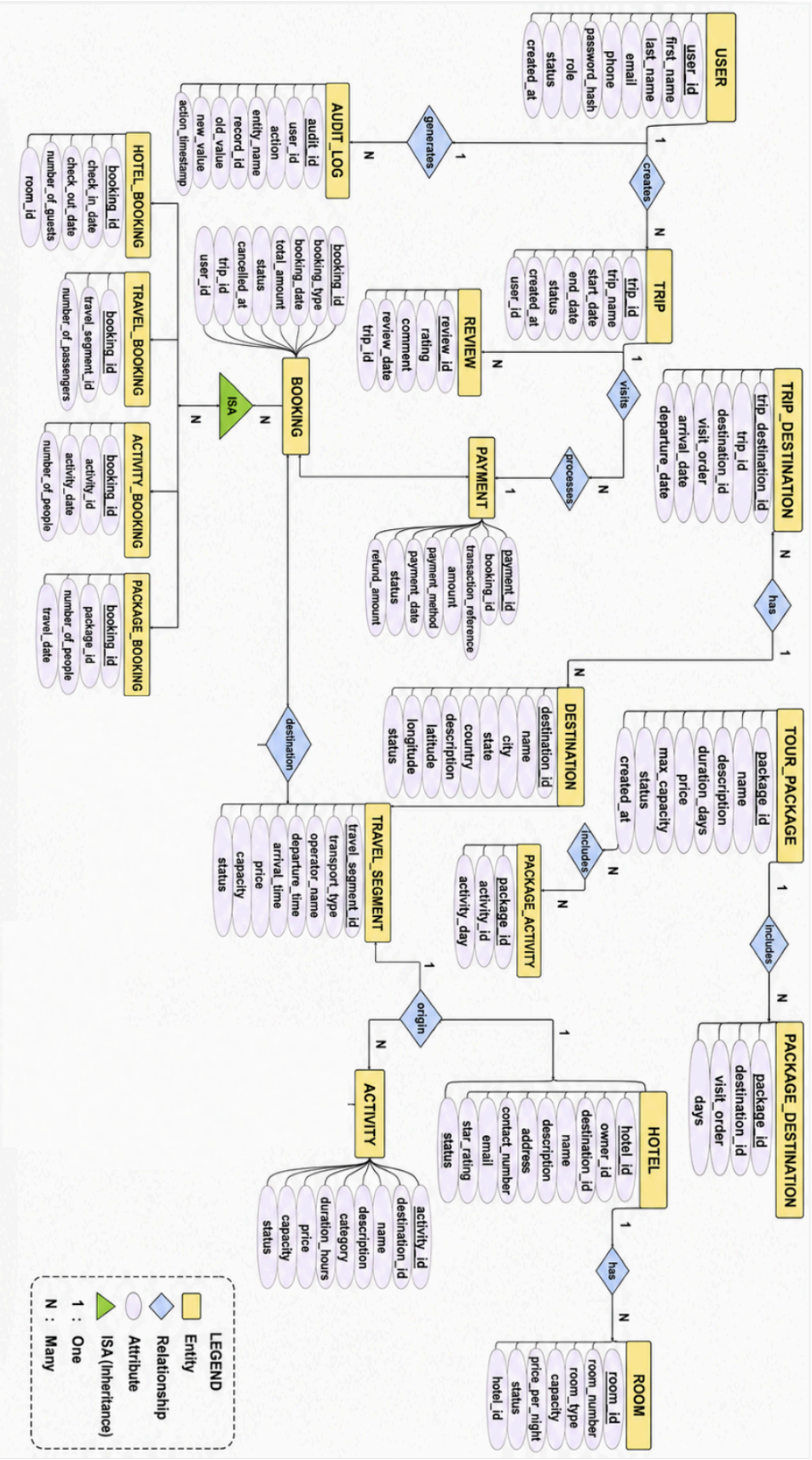
In this project the Integrated Tourism Booking and Travel Management System covers all the centralized management's of tourism related information and travel (like hotel booking, activities, the hotel information etc) transactions through a relational database. The system is intended to support majorly 3 users, the customers, service providers, and administrators in managing and accessing tourism services and booking information.

The major functionalities of the system include:

- **Destination Management:** Storing and managing the information about tourist destinations, including destination details and related tourism services.
- **Hotel Management:** Maintaining the hotel information, room details, room types, pricing, and availability.
- **Activity Management:** Managing tourism activities associated with destinations, including activity details, schedules, and pricing.
- **Tour Package:** Storing and managing tour packages that combine destinations, accommodations and activities.
- **Customer Management:** Maintaining customer profiles and their travel related information.
- **Booking Management:** Creating and managing bookings for hotels, rooms, activities, and tour packages, along with booking dates, status, and associated customer details.
- **Availability Management:** Maintaining and updating the availability of rooms, activities, and other bookable tourism services to reduce the booking conflicts.
- **Cancellation Management:** Managing booking cancellations and returning the corresponding booking payment records.
- **Review Management:** Storing customer reviews and ratings for tourism services such as hotels, activities, and destinations.
- **Administrative Management:** Providing administrators with the ability to manage the tourism services, customer records, bookings, payments.
- **Reporting and Data Retrieval:** Supporting SQL based retrieval of database information.

From a database perspective, the scope includes designing and implementing an interconnected but independent relationship between customers, destinations, hotels, rooms, activities, tour packages, bookings, payments, and reviews. The system will be using the appropriate primary keys, foreign keys, integrity constraints, normalization, SQL queries, and database programming features to ensure data consistency, integrity, and efficient retrieval. The system focuses mainly primarily on the database design and management of tourism booking operations

IV. Database Design



Yes, this dataset uses normalization to avoid redundancy.

Primary keys and foreign keys used : audit_id, trip_id, booking_id, review_id, trip_destination_id, payment_id, package_id, travel_segment_id, destination_id, activity_id, room_id, hotel_id, user_id

Some of the integrity constraints include not null, unique, check, primary key and foreign key constraints.

V. Relational Schema

Tourism Booking & Travel Management Portal is a relational database system designed to manage the main operations involved in tourism, travel planning, and online booking. The database is organized into 19 tables covering identity management, tourism catalog information, trip and itinerary management, bookings and payments, and audit tracking. The schema is designed to represent the different relationships between users, destinations, hotels, rooms, activities, travel segments, tour packages, trips, and bookings while maintaining data integrity through primary and foreign keys.

The database begins with the core entities Users, Destinations, and TourPackages, which provide the foundation for the rest of the system. Users can own hotels, create trips, make bookings, and perform actions recorded in the audit logs. Destinations are associated with hotels, activities, and travel segments, while tour packages can contain multiple destinations and activities. Trips are associated with users and can include multiple destinations as part of an itinerary. The schema also supports different types of tourism bookings through the Bookings supertype and its specialized subtypes: HotelBookings, TravelBookings, ActivityBookings, and PackageBookings. This allows common booking information to be stored in the parent table while specific information for each booking type is maintained in its corresponding child table. Payments are linked to bookings, and reviews provide a one-to-one relationship with trips.

The following schema defines the complete relational structure, including the entities, attributes, primary keys, foreign keys, one-to-many relationships, many-to-many junction tables, one-to-one relationship, and supertype/subtype design used by the Tourism Booking & Travel Management Portal.

1 Base entities (no FK dependencies)

The base entities form the foundation of the database and do not depend on other tables through foreign keys. These entities store the primary information about users, destinations, and tour packages. Each entity has its own primary key, which uniquely identifies each record and can be referenced by other tables.

- Users(user_id PK, first_name, last_name, email UNIQUE, phone, password_hash, role, status, created_at)
- Destinations(destination_id PK, name, city, state, country, description, latitude, longitude, status)
- TourPackages(package_id PK, name, description, duration_days, price, max_capacity, status, created_at)

2 1:M / M:1 relationships (2 tables each)

The following entities represent one-to-many (1:M) and many-to-one (M:1) relationships within the system. In these relationships, the foreign key is placed in the table on the “many” side to establish the connection with the parent entity. These relationships allow users, destinations, trips, bookings, hotels, rooms, activities, travel segments, payments, and audit records to be connected while maintaining referential integrity.

- **Users** → **Hotels** (owner) and **Destinations** → **Hotels** Hotels(hotel_id PK, owner_id FK → Users, destination_id FK → Destinations, name, description, address, contact_number, email, star_rating, status)
- **Hotels** → **Rooms** Rooms(room_id PK, room_number, room_type, capacity, price_per_night, status, hotel_id FK → Hotels)
- **Destinations** → **Activities** Activities(activity_id PK, destination_id FK → Destinations, name, description, category, duration_hours, price, capacity, status)
- **Users** → **Trips** Trips(trip_id PK, trip_name, start_date, end_date, status, created_at, user_id FK → Users)
- **Destinations** → **TravelSegments**, **twice** (origin and destination are two separate M:1 links into the same table — not M:N, since each segment has exactly one of each) TravelSegments(travel_segment_id PK, transport_type, operator_name, departure_time, arrival_time, price, capacity, status, origin_destination_id FK → Destinations, destination_destination_id FK → Destinations)
- **Users** → **Bookings** and **Trips** → **Bookings** Bookings(booking_id PK, booking_type, booking_date, total_amount, status, cancelled_at, trip_id FK → Trips, user_id FK → Users)
- **Bookings** → **Payments** Payments(payment_id PK, booking_id FK → Bookings, transaction_reference UNIQUE, amount, payment_method, payment_date, status, refund_amount)
- **Users** → **AuditLogs** AuditLogs(audit_id PK, action, entity_name, record_id, old_value, new_value, action_timestamp, user_id FK → Users)

3 1:1 relationship (2 tables)

The one-to-one (1:1) relationship is used where each record in one entity can be associated with at most one record in another entity.

Trips ↔ **Reviews** Reviews(review_id PK, rating, comment, review_date, trip_id FK UNIQUE → Trips)

4 M:N relationships — 3 tables each

Many-to-many (M:N) relationships occur when multiple records from one entity can be associated with multiple records from another entity. These relationships cannot be represented directly using only foreign keys in the two original tables, so separate junction tables are introduced.

TourPackages ↔ **Destinations**

- TourPackages(package_id PK, ...) — already defined in Base entities
- Destinations(destination_id PK, ...) — already defined in Base entities

- PackageDestinations(package_id PK FK -> TourPackages, destination_id PK FK -> Destinations, visit_order, days) ← junction

TourPackages ↔ Activities

- TourPackages(package_id PK, ...) — already defined in Base entities
- Activities(activity_id PK, ...) — already defined in 1:M / M:1 relationships
- PackageActivities(package_id PK FK -> TourPackages, activity_id PK FK -> Activities, activity_day) ← junction

Trips ↔ Destinations

- Trips(trip_id PK, ...) — already defined in 1:M / M:1 relationships
- Destinations(destination_id PK, ...) — already defined in Base entities
- TripDestinations(trip_destination_id PK, trip_id FK -> Trips, destination_id FK -> Destinations, visit_order, arrival_date, departure_date) ← junction (surrogate PK here instead of composite, since arrival/departure dates and order hang off the pairing itself)

5 Supertype/Subtype (ISA) — 1 parent + 4 children = 5 tables

The supertype/subtype (ISA) structure is used to represent different types of bookings that share common attributes.

Bookings(booking_id PK, booking_type, booking_date, total_amount, status, cancelled_at, trip_id FK -> Trips, user_id FK -> Users) ← **parent**

Each child shares the parent's PK as its own PK *and* FK (1:1 with Bookings), plus one extra FK pointing to whatever is actually being booked:

HotelBookings(booking_id PK FK -> Bookings, check_in_date, check_out_date, number_of_guests, room_id FK -> Rooms)

TravelBookings(booking_id PK FK -> Bookings, number_of_passengers, travel_segment_id FK -> TravelSegments)

ActivityBookings(booking_id PK FK -> Bookings, activity_date, number_of_people, activity_id FK -> Activities)

PackageBookings(booking_id PK FK -> Bookings, number_of_people, travel_date, package_id FK -> TourPackages)

Tourism Booking & Travel Management Portal — Oracle SQL schema. Nineteen tables covering identity, catalog, itinerary, booking, and audit concerns.

5.1 App_User

Central identity table for every actor in the portal — travelers, hotel owners, and administrators — distinguished by role. Nearly every other table ultimately traces back to this table as the acting or owning user.

Table: App_User

Field	Type	Null	Key	Default	Extra
user_id	NUMBER	NO	PK	NULL	Sequence-generated
first_name	VARCHAR2(50)	NO	-	NULL	-
last_name	VARCHAR2(50)	NO	-	NULL	-
email	VARCHAR2(100)	NO	UQ	NULL	-
phone	VARCHAR2(15)	YES	UQ	NULL	-
password_hash	VARCHAR2(255)	NO	-	NULL	-
role	VARCHAR2(20)	NO	-	NULL	-
status	VARCHAR2(20)	NO	-	NULL	-
created_at	TIMESTAMP	NO	-	CURRENT_TIMESTAMP	-

5.2 Destination

Master geography table shared across Hotel, Activity, Travel_Segment, Trip, and Tour_Package, so a place is defined once and referenced everywhere instead of being re-typed as a string on each row.

Table: Destination

Field	Type	Null	Key	Default	Extra
destination_id	NUMBER	NO	PK	NULL	Sequence-generated
name	VARCHAR2(100)	NO	-	NULL	-
city	VARCHAR2(100)	NO	-	NULL	-
state	VARCHAR2(100)	YES	-	NULL	-
country	VARCHAR2(100)	NO	-	NULL	-
description	CLOB	YES	-	NULL	-
latitude	NUMBER(9,6)	YES	-	NULL	-
longitude	NUMBER(9,6)	YES	-	NULL	-
status	VARCHAR2(20)	NO	-	NULL	-

5.3 Hotel

Property record owned by an App_User (in the hotel-owner role) and anchored to a Destination. Parent of Room, and the entity summarized whenever a Hotel_Booking is made.

Table: Hotel

Field	Type	Null	Key	Default	Extra
hotel_id	NUMBER	NO	PK	NULL	Sequence-generated
owner_id	NUMBER	NO	FK	NULL	-
destination_id	NUMBER	NO	FK	NULL	-
name	VARCHAR2(150)	NO	-	NULL	-
description	CLOB	YES	-	NULL	-
address	VARCHAR2(255)	NO	-	NULL	-
contact_number	VARCHAR2(15)	YES	-	NULL	-
email	VARCHAR2(100)	YES	-	NULL	-
star_rating	NUMBER(2,1)	YES	-	NULL	-
status	VARCHAR2(20)	NO	-	NULL	-

5.4 Room

Bookable inventory unit belonging to a single Hotel. Referenced by Hotel_Booking to record exactly which room was reserved for a given stay.

Table: Room

Field	Type	Null	Key	Default	Extra
room_id	NUMBER	NO	PK	NULL	Sequence-generated
hotel_id	NUMBER	NO	FK	NULL	-
room_number	VARCHAR2(20)	NO	-	NULL	-
room_type	VARCHAR2(30)	NO	-	NULL	-
capacity	NUMBER	NO	-	NULL	-
price_per_night	NUMBER(10,2)	NO	-	NULL	-
status	VARCHAR2(20)	NO	-	NULL	-

5.5 Activity

Bookable experience tied to a Destination. Referenced directly by Activity_Booking, and indirectly through Package_Activity when bundled into a tour package.

Table: Activity

Field	Type	Null	Key	Default	Extra
activity_id	NUMBER	NO	PK	NULL	Sequence-generated

destination_id	NUMBER	NO	FK	NULL	-
name	VARCHAR2(150)	NO	-	NULL	-
description	CLOB	YES	-	NULL	-
category	VARCHAR2(50)	YES	-	NULL	-
duration_hours	NUMBER(4,2)	YES	-	NULL	-
price	NUMBER(10,2)	NO	-	NULL	-
capacity	NUMBER	YES	-	NULL	-
status	VARCHAR2(20)	NO	-	NULL	-

5.6 Tour_Package

Pre-built multi-day itinerary that bundles Destinations (via Package_Destination) and Activities (via Package_Activity) and is sold as a single product through Package_Booking.

Table: Tour_Package

Field	Type	Null	Key	Default	Extra
package_id	NUMBER	NO	PK	NULL	Sequence-generated
name	VARCHAR2(150)	NO	-	NULL	-
description	CLOB	YES	-	NULL	-
duration_days	NUMBER	NO	-	NULL	-
price	NUMBER(10,2)	NO	-	NULL	-
max_capacity	NUMBER	YES	-	NULL	-
status	VARCHAR2(20)	NO	-	NULL	-
created_at	TIMESTAMP	NO	-	CURRENT_TIMESTAMP	-

5.7 Travel_Segment

A single point-to-point transport leg (flight, train, bus, ferry, etc.) between two Destinations. Referenced by Travel_Booking whenever a traveler reserves passage on that leg.

Table: Travel_Segment

Field	Type	Null	Key	Default	Extra
travel_segment_id	NUMBER	NO	PK	NULL	Sequence-generated
origin_destination_id	NUMBER	NO	FK	NULL	-

destination_destination_id	NUMBER	NO	FK	NULL	-
transport_type	VARCHAR2(30)	NO	-	NULL	-
operator_name	VARCHAR2(100)	YES	-	NULL	-
departure_time	TIMESTAMP	NO	-	NULL	-
arrival_time	TIMESTAMP	NO	-	NULL	-
price	NUMBER(10,2)	NO	-	NULL	-
capacity	NUMBER	YES	-	NULL	-
status	VARCHAR2(20)	NO	-	NULL	-

5.8 Trip

A traveler's overall journey container. Groups the Destinations visited (via Trip_Destination) and the Bookings and single Review made against the journey.

Table: Trip

Field	Type	Null	Key	Default	Extra
trip_id	NUMBER	NO	PK	NULL	Sequence-generated
user_id	NUMBER	NO	FK	NULL	-
trip_name	VARCHAR2(150)	NO	-	NULL	-
start_date	DATE	NO	-	NULL	-
end_date	DATE	NO	-	NULL	-
status	VARCHAR2(20)	NO	-	NULL	-
created_at	TIMESTAMP	NO	-	CURRENT_TIMESTAMP	-

5.9 Trip_Destination

Junction table sequencing which Destinations a Trip visits and in what order, with per-stop arrival and departure dates. Composite primary key (trip_id, destination_id).

Table: Trip_Destination

Field	Type	Null	Key	Default	Extra
trip_id	NUMBER	NO	PK, FK	NULL	-
destination_id	NUMBER	NO	PK, FK	NULL	-
visit_order	NUMBER	NO	-	NULL	-
arrival_date	DATE	YES	-	NULL	-

departure_date	DATE	YES	-	NULL	-
----------------	------	-----	---	------	---

5.10 Package_Destination

Junction table defining which Destinations a Tour_Package covers, their visiting order, and how many days are allotted to each stop. Composite primary key (package_id, destination_id).

Table: Package_Destination

Field	Type	Null	Key	Default	Extra
package_id	NUMBER	NO	PK, FK	NULL	-
destination_id	NUMBER	NO	PK, FK	NULL	-
visit_order	NUMBER	YES	-	NULL	-
days	NUMBER	YES	-	NULL	-

5.11 Package_Activity

Junction table listing which Activities are bundled into a Tour_Package and on which day of the itinerary each one occurs. Composite primary key (package_id, activity_id).

Table: Package_Activity

Field	Type	Null	Key	Default	Extra
package_id	NUMBER	NO	PK, FK	NULL	-
activity_id	NUMBER	NO	PK, FK	NULL	-
activity_day	NUMBER	YES	-	NULL	-

5.12 Booking

The central polymorphic fact table for every purchase in the system. Each row is extended by exactly one of Hotel_Booking, Package_Booking, Activity_Booking, or Travel_Booking, and settled by one or more Payments.

Table: Booking

Field	Type	Null	Key	Default	Extra
booking_id	NUMBER	NO	PK	NULL	Sequence-generated
user_id	NUMBER	NO	FK	NULL	-
trip_id	NUMBER	YES	FK	NULL	-
booking_type	VARCHAR2(20)	NO	-	NULL	CHECK IN ('HOTEL','PACKAGE','ACTIVITY','TRAVEL')

booking_date	TIMESTAMP	NO	-	CURRENT_TIMESTAMP	-
total_amount	NUMBER(10,2)	NO	-	NULL	-
status	VARCHAR2(20)	NO	-	NULL	-
cancelled_at	TIMESTAMP	YES	-	NULL	-

5.13 Hotel_Booking

Subtype of Booking that stores hotel-specific stay details — which Room, check-in/out dates, and guest count — for rows where booking_type = 'HOTEL'.

Table: Hotel_Booking

Field	Type	Null	Key	Default	Extra
booking_id	NUMBER	NO	PK, FK	NULL	-
room_id	NUMBER	NO	FK	NULL	-
check_in_date	DATE	NO	-	NULL	-
check_out_date	DATE	NO	-	NULL	-
number_of_guests	NUMBER	NO	-	NULL	-

5.14 Package_Booking

Subtype of Booking that stores tour-package-specific details — party size and travel date — for rows where booking_type = 'PACKAGE'.

Table: Package_Booking

Field	Type	Null	Key	Default	Extra
booking_id	NUMBER	NO	PK, FK	NULL	-
package_id	NUMBER	NO	FK	NULL	-
number_of_people	NUMBER	NO	-	NULL	-
travel_date	DATE	NO	-	NULL	-

5.15 Activity_Booking

Subtype of Booking that stores activity-specific details — the activity date and party size — for rows where booking_type = 'ACTIVITY'.

Table: Activity_Booking

Field	Type	Null	Key	Default	Extra
booking_id	NUMBER	NO	PK, FK	NULL	-
activity_id	NUMBER	NO	FK	NULL	-
activity_date	DATE	NO	-	NULL	-
number_of_people	NUMBER	NO	-	NULL	-

5.16 Travel_Booking

Subtype of Booking that stores transport-specific details — which Travel_Segment and how many passengers — for rows where booking_type = 'TRAVEL'.

Table: Travel_Booking

Field	Type	Null	Key	Default	Extra
booking_id	NUMBER	NO	PK, FK	NULL	-
travel_segment_id	NUMBER	NO	FK	NULL	-
number_of_passengers	NUMBER	NO	-	NULL	-

5.17 Payment

Financial settlement record tied to a Booking. Tracks the transaction reference, method, status, and any amount refunded back to the traveler.

Table: Payment

Field	Type	Null	Key	Default	Extra
payment_id	NUMBER	NO	PK	NULL	Sequence-generated
booking_id	NUMBER	NO	FK	NULL	-
transaction_reference	VARCHAR2(100)	NO	UQ	NULL	-
amount	NUMBER(10,2)	NO	-	NULL	-
payment_method	VARCHAR2(30)	NO	-	NULL	-
payment_date	TIMESTAMP	NO	-	CURRENT_TIMESTAMP	-
status	VARCHAR2(20)	NO	-	NULL	-
refund_amount	NUMBER(10,2)	NO	-	0	-

5.18 Review

Traveler feedback left against a completed Trip. The UNIQUE constraint on trip_id ensures one Trip produces at most one Review.

Table: Review

Field	Type	Null	Key	Default	Extra
review_id	NUMBER	NO	PK	NULL	Sequence-generated
trip_id	NUMBER	NO	FK, UQ	NULL	-
rating	NUMBER	NO	-	NULL	CHECK BETWEEN 1 AND 5
comment	CLOB	YES	-	NULL	-
review_date	TIMESTAMP	NO	-	CURRENT_TIMESTAMP	-

5.19 Audit_Log

System-wide change-tracking table recording who did what to which record, and the before/after values, across the entire schema — used for compliance review and debugging.

Table: Audit_Log

Field	Type	Null	Key	Default	Extra
audit_id	NUMBER	NO	PK	NULL	Sequence-generated
user_id	NUMBER	YES	FK	NULL	-
action	VARCHAR2(50)	NO	-	NULL	-
entity_name	VARCHAR2(50)	NO	-	NULL	-
record_id	NUMBER	YES	-	NULL	-
old_value	CLOB	YES	-	NULL	-
new_value	CLOB	YES	-	NULL	-
action_timestamp	TIMESTAMP	NO	-	CURRENT_TIMESTAMP	-

VI. Database Technologies

The Tourism Management Database System is implemented using Oracle Database, which serves as the Relational Database Management System (RDBMS). SQL is used for creating and modifying database structures, inserting and managing records, retrieving information, performing joins and subqueries, and generating reports.

PL/SQL is used to implement database-level operations and business logic. The system can incorporate stored procedures, functions, triggers, and cursors to automate operations, enforce business rules, and process database records efficiently. Primary keys, foreign keys, and integrity constraints are also implemented to maintain data accuracy and consistency.

Technologies Used

- **DBMS:** Oracle Database
- **Database Model:** Relational Database Management System (RDBMS)
- **Query Language:** SQL
- **Procedural Language:** PL/SQL
- **SQL Features:** DDL, DML, DQL, joins, subqueries, aggregate functions, views
- **PL/SQL Features:** Procedures, functions, triggers, cursors
- **Integrity Mechanisms:** Primary keys, foreign keys, **NOT NULL**, **UNIQUE**, and **CHECK** constraints
- **Transaction Management:** **COMMIT**, **ROLL**
- **BACK**, and **SAVEPOINT** for maintaining transaction consistency

VII. Frontend

In this project we will use HTML/CSS/JavaScript to develop a web based frontend that provides an user friendly interactive interface for customers, service providers, and administrators.

- HTML will be used to structure webpages and forms.
- CSS will be used for styling and layout
- JavaScript will provide client side interactivity, form validation, and dynamic interaction with the system.
- Python with the Flask framework will be used as the backend or the application layer to handle requests from the frontend and communicate with the Oracle Database. The frontend will interact with the database through this backend layer, which will execute the required SQL operations and retrieve or update the tourism related information such as destinations, hotels, rooms, activities, tour packages, customers, bookings, payments, and reviews.

The overall interaction will follow:

HTML/CSS/JavaScript → Python + Flask → Oracle Database

For example, when a customer searches for an available hotel or makes a booking, the frontend will collect the required information and send it to the Flask backend. The backend will process the request and communicate with Oracle using SQL or the database operations. The retrieved or updated information will then be returned to the frontend and displayed to the

user. This allows the frontend to provide a user friendly interface while the Oracle database remains responsible for the centralized storage, management, integrity, and retrieval of tourism related data.

VIII. Expected Outcome

The proposed system is expected to provide a centralized and reliable platform for managing tourism-related data and booking transactions. It will enable efficient management of destinations, hotels, rooms, activities, tour packages, customers, bookings, payments, and reviews while reducing data redundancy and maintaining data integrity and consistency.

The system is expected to:

- **Improve data organization and accessibility** by maintaining tourism information in a structured relational database.
- **Reduce data redundancy and inconsistencies** through proper database design and normalization.
- **Minimize booking conflicts** by maintaining accurate availability and reservation records.
- **Improve transaction reliability** by ensuring consistency during booking, payment, cancellation, and related operations.
- **Enable efficient information retrieval** through SQL queries, views, and database operations.
- **Support administrative decision-making** through reports and tourism-related booking and revenue analysis.
- **Provide secure and role-based access** for customers, service providers, and administrators.
- **Provide a user-friendly interface** for searching tourism services, making bookings, managing trips, and accessing booking information.
- **Demonstrate the practical application of DBMS concepts** including relational modeling, normalization, integrity constraints, transactions, procedures, functions, triggers, and complex SQL queries.

Overall, the expected outcome is a scalable and efficient tourism management system that improves the accuracy, consistency, and accessibility of tourism data while providing an integrated platform for managing the complete booking lifecycle